

SIX WEEKS INDUSTRIAL TRAINING REPORT

On

‘SIGNLEXICON: SIGN LANGUAGE RECOGNITION SYSTEM USING COMPUTER VISION AND DEEP LEARNING’

AT

‘SOLITAIRE INFOSYSTEMS PVT. LTD.’

BY

MANNATDEEP KAUR CHOHAN

ROLL NO. 28422405159



**DEPARTMENT OF MECHANICAL ENGINEERING
GURU NANAK DEV UNIVERSITY, AMRITSAR**

‘JULY’ 26’

GURU NANAK DEV UNIVERSITY, AMRITSAR

CANDIDATE'S DECLARATION

I, 'MANNATDEEP KAUR CHOCHAN', hereby declare that I have undertaken six (06) weeks of Industry Oriented Project Training at 'SOLITAIRE INFOSYSTEMS PVT. LTD.' during a period from 15 June'26 to 29 July'26 in partial fulfilment of requirements for the award of degree of B. Tech (Mechanical Engineering) at GURU NANAK DEV UNIVERSITY, AMRITSAR. The work which is being presented in the training report submitted to Department of Mechanical Engineering, Guru Nanak Dev University, Amritsar is an authentic record of training work

Signature of Student

CERTIFICATE

ABSTRACT

Communication between deaf and hearing individuals remains a significant challenge in modern society. Sign language serves as an effective means of communication for the deaf and mute community; however, most people are unable to understand sign language gestures. With the advancement of Artificial Intelligence, Computer Vision, and Deep Learning technologies, it has become possible to develop intelligent systems capable of recognizing hand gestures automatically.

The project entitled **“SIGNLEXICON: Sign Language Recognition System Using Computer Vision And Deep Learning”** was developed during the six-week industrial training at Solitaire Infosystems Pvt. Ltd. The primary objective of this project is to recognize American Sign Language (ASL) alphabets using a Convolutional Neural Network (CNN) and provide real-time predictions through a user-friendly desktop application.

The system was trained using the ASL Alphabet Dataset consisting of multiple gesture classes. Image preprocessing techniques such as resizing and normalization were applied before training the model. A CNN-based architecture was developed using TensorFlow and Keras frameworks. The trained model was integrated into a graphical user interface developed using Tkinter. The application supports both image upload and real-time webcam-based gesture recognition. The developed system successfully predicts sign language alphabets and displays the prediction along with confidence scores. The project demonstrates the practical application of Deep Learning and Computer Vision techniques in developing assistive technologies for improving communication accessibility.

ACKNOWLEDGEMENT

I express my sincere gratitude to **Solitaire Infosystems Pvt. Ltd., Mohali**, for providing me the opportunity to undergo six weeks industrial training and gain practical exposure to emerging technologies in the field of Artificial Intelligence and Machine Learning.

I would like to thank my industry mentor and all members of the development team for their valuable guidance, continuous support, and technical assistance throughout the training period. Their suggestions and encouragement helped me successfully complete the project entitled **“SIGNLEXICON: Sign Language Recognition System Using Computer Vision and Deep Learning.”**

I am also thankful to the faculty members of the Department of Mechanical Engineering, Guru Nanak Dev University, Amritsar, for their guidance and motivation during the training period.

Finally, I would like to express my heartfelt gratitude to my parents and friends for their constant support, encouragement, and inspiration throughout the completion of this training and project work.

Mannatdeep Kaur Chohan

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iv
	ACKNOWLEDGEMENT	v
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
I.	INTRODUCTION	1
	1.1 Introduction	1
	1.2 Necessity of the Project	2
	1.3 Objectives	2
	1.4 Theme of the Project	3
	1.5 Organization of the Report	4
II.	LITERATURE SURVEY	5
	2.1 Introduction	5
	2.2 Review of Existing Sign Language Recognition Systems	6
	2.3 Limitations of Existing Systems	7
	2.4 Research Gap	9
	2.5 Summary	10
III.	TRAINING WORK	11
	3.1 Introduction	11
	3.2 Company Profile	12
	3.3 Training Schedule	12
	3.4 Project Selection	13
	3.5 Development Methodology	13
	3.6 Dataset Collection	14
	3.7 Image Preprocessing	16
	3.8 CNN Model Development	16
	3.9 Model Training	17
	3.10 Desktop Application	18
	3.11 Webcam Recognition Module	19
	3.12 Technologies Used	19
	3.13 Training Outcome	20
	3.14 Summary	21
IV.	EVALUATION OF TRAINING	22
	4.1 Introduction	22
	4.2 Testing Methodology	22
	4.3 Functional Testing	23
	4.4 Experimental Results	23
	4.5 Performance Evaluation	26
	4.6 Sample Prediction Results	26
	4.7 Discussion	27
	4.8 Challenges Faced During Training	28

	4.9 Skills Acquired During Training	29
	4.10 Summary	29
V.	CONCLUSION & FUTURE SCOPE OF TRAINING	30
	5.1 Introduction	30
	5.2 Conclusion	30
	5.3 Learning Outcomes	31
	5.4 Limitations of the Proposed System	32
	5.5 Future Scope	32
	5.6 Industrial Training Experience	33
	5.7 Final Conclusion	34
	5.8 Summary	35

LIST OF TABLES

TABLE NO.	TITLE
Table 2.1	Review of Existing Sign Language Recognition Systems.....6
Table 2.2	Drawbacks of Existing Systems.....8
Table 3.1	Training Schedule.....12
Table 3.2	Training Parameters.....18
Table 3.3	Software and Technologies Used.....20
Table 4.1	Functional Testing Results.....23
Table 4.2	Performance Evaluation.....26
Table 4.3	Sample Prediction Results.....26
Table A.1	Hardware Requirements.....36
Table A.2	Software Requirements.....36

LIST OF FIGURES

TABLE NO.	TITLE
Figure 2.1	Evolution of Sign Language Recognition Techniques.....8
Figure 3.1	Development Methodology.....13
Figure 3.2	ASL Alphabet Dataset.....15
Figure 3.3	CNN Model Architecture.....17
Figure 3.4	SIGNLEXICON Desktop Application.....19
Figure 4.1	Home Screen of SIGNLEXICON.....24
Figure 4.2	Image Upload Prediction.....25
Figure 4.3	Webcam Recognition.....25
Figure 4.4	Sample Prediction Results.....27

LIST OF ABBREVIATIONS

ABBREVIATION	DESCRIPTION
AI	Artificial Intelligence
CNN	Convolutional Neural Network
ASL	American Sign Language
ISL	Indian Sign Language
GUI	Graphical User Interface
ML	Machine Learning
DL	Deep Learning
RGB	Red, Green and Blue
PIL	Python Imaging Library
API	Application Programming Interface
CPU	Central Processing Unit
IDE	Integrated Development Environment
OpenCV	Open Source Computer Vision Library
ROI	Region of Interest
TensorFlow	Open Source Machine Learning Framework

CHAPTER I

INTRODUCTION

1.1 Introduction

Sign language is one of the most effective methods of communication for individuals who are deaf or hard of hearing. It uses hand gestures, facial expressions, and body movements to convey information and emotions. Although sign language enables effective communication within the deaf community, many people who are unfamiliar with it face difficulties in understanding these gestures. This communication gap often creates challenges in education, workplaces, healthcare, public services, and everyday interactions. Recent advancements in Artificial Intelligence (AI), Computer Vision, and Deep Learning have made it possible to develop intelligent systems capable of recognizing visual patterns with high accuracy. Computer Vision enables machines to capture and interpret images or videos, while Deep Learning models, particularly Convolutional Neural Networks (CNNs), have demonstrated excellent performance in image classification and object recognition tasks. These technologies provide an opportunity to develop automated sign language recognition systems that can assist in bridging the communication gap between deaf and hearing individuals.

The project "**SIGNLEXICON: Sign Language Recognition System Using Computer Vision and Deep Learning**" was developed during the six-week industrial training at **Solitaire Infosystems Pvt. Ltd., Mohali**. The system is designed to recognize American Sign Language (ASL) alphabet gestures from both uploaded images and real-time webcam input. A CNN model was trained using the ASL Alphabet Dataset to classify hand gestures into their corresponding alphabet classes. The trained model was then integrated into a desktop application developed using Python and Tkinter to provide a simple and user-friendly interface.

The application allows users to upload an image containing a hand gesture or use a webcam for live recognition. After processing the input image, the trained CNN predicts the corresponding alphabet and displays the prediction along with its confidence score. This project demonstrates the practical implementation of Deep Learning and Computer Vision techniques in developing assistive technologies that promote accessibility and inclusive communication.

1.2 Necessity of the Project

Communication is an essential part of human life; however, individuals with hearing or speech impairments often face significant barriers while interacting with people who do not understand sign language. In many situations, such as educational institutions, hospitals, government offices, workplaces, and public places, the absence of sign language interpreters limits effective communication and reduces accessibility. The rapid growth of Artificial Intelligence and Deep Learning has opened new possibilities for developing automated systems that can recognize sign language accurately and efficiently. An intelligent sign language recognition system can help bridge the communication gap by translating hand gestures into understandable text in real time. The development of **SIGNLEXICON** is motivated by the need to provide an accessible, reliable, and user-friendly solution for recognizing sign language alphabets. By integrating Computer Vision with Deep Learning, the system minimizes manual interpretation and provides instant predictions through a graphical desktop interface. Such applications can contribute significantly to educational support, assistive communication, and future accessibility solutions.

1.3 Objectives of the Project

The primary objectives of the SIGNLEXICON project are:

- To develop an intelligent sign language recognition system using Deep Learning techniques.
- To train a Convolutional Neural Network (CNN) model capable of recognizing American Sign Language (ASL) alphabet gestures.
- To preprocess and classify hand gesture images accurately using Computer Vision techniques.
- To develop a user-friendly desktop application using Python and Tkinter.
- To provide prediction results along with confidence scores for better interpretation.
- To support both image-based recognition and real-time webcam recognition.
- To demonstrate the practical application of Artificial Intelligence in assistive communication systems.

1.4 Theme of the Project

The theme of this project is the application of **Artificial Intelligence, Computer Vision, and Deep Learning** for improving communication accessibility. The project combines image processing techniques with a Convolutional Neural Network to recognize hand gestures representing American Sign Language alphabets. The developed desktop application demonstrates how intelligent systems can assist individuals with hearing and speech impairments by providing quick and accurate gesture recognition. The project also highlights the practical integration of machine learning models with graphical user interfaces to create interactive and user-friendly applications. The proposed system serves as a foundation for

future developments such as sentence recognition, speech generation, and mobile-based sign language translation systems.

1.5 Organization of the Report

This report is organized into five chapters, each covering a specific aspect of the industrial training and project development.

- **Chapter I** introduces the project, explains its necessity, objectives, theme, and overall organization.
- **Chapter II** presents the literature survey of existing sign language recognition systems and related research carried out in the field of Computer Vision and Deep Learning.
- **Chapter III** describes the work carried out during the industrial training, including the company profile, software and hardware requirements, dataset preparation, model development, application development, and system implementation.
- **Chapter IV** discusses the experimental results, testing methodology, performance evaluation, and analysis of the developed system.
- **Chapter V** concludes the report by summarizing the project outcomes, applications, limitations, and future scope of the proposed system.

CHAPTER II

LITERATURE SURVEY

2.1 Introduction

The field of sign language recognition has experienced significant growth over the past decade due to advancements in Artificial Intelligence (AI), Machine Learning (ML), Computer Vision, and Deep Learning. Researchers across the world have developed various techniques to recognize sign language gestures using image processing, sensor-based devices, wearable gloves, and vision-based systems. These systems aim to reduce communication barriers between deaf or hard-of-hearing individuals and the hearing community by translating sign language into readable text or speech.

Earlier sign language recognition systems mainly relied on handcrafted image features and traditional machine learning algorithms. Although these methods provided moderate accuracy, they were sensitive to variations in lighting conditions, background noise, and hand orientation. With the emergence of Deep Learning, Convolutional Neural Networks (CNNs) have become one of the most effective approaches for image classification tasks due to their ability to automatically extract important visual features from images.

In this project, an extensive literature survey was carried out to understand existing sign language recognition techniques, their advantages, limitations, and practical applications. The findings from this survey helped in selecting an appropriate CNN-based approach for the development of the SIGNLEXICON desktop application.

2.2 Review of Existing Sign Language Recognition Systems

Several sign language recognition systems have been developed using different datasets and deep learning techniques. Most modern approaches utilize Computer Vision combined with Deep Learning models to recognize static or dynamic hand gestures. The following table summarizes some of the major techniques, datasets, models, improvements, and their reported accuracy.

Table 2.1 Review of Existing Sign Language Recognition Systems

TECHNIQUE/MODEL	DATA USED	DEEP LEARNING MODEL	KEY IMPROVEMENT	ACCURACY
CNN Based Sign language recognition	ASL alphabet dataset	CNN	Apply data augmentation, use high resolution cameras and real time processing.	96-98%
CNN + LSTM Model	RWTH-PHOENIX Weather 2014	CNN + LSTM	Use lightweight models and attention mechanisms.	90-94%
Media Pipe + CNN	Custom dataset	Media Pipe Hands + CNN	Increase dataset size and support dynamic gesture recognition.	97-99%
YOLO + CNN	Custom Real – Time Dataset	YOLOv5 + CNN	Improve object tracking and multi hand detection.	95-97%

Transfer Learning Approach	ASL alphabet dataset	MobileNet, Resnet50	Fine tune on larger datasets and optimize for edge/mobile devices.	98-99%
Vision Transformer (ViT)	WLASL dataset	ViT	Hybrid CNN - ViT architecture and model compression.	98-99%

The comparison presented in Table 2.1 shows that recent Deep Learning models achieve very high recognition accuracy. CNN-based approaches provide reliable performance for static gesture recognition, whereas hybrid models such as CNN-LSTM and Vision Transformers improve recognition of complex gestures. Despite these improvements, several practical limitations still exist, especially in terms of computational requirements and real-time deployment.

2.3 Limitations of Existing Systems

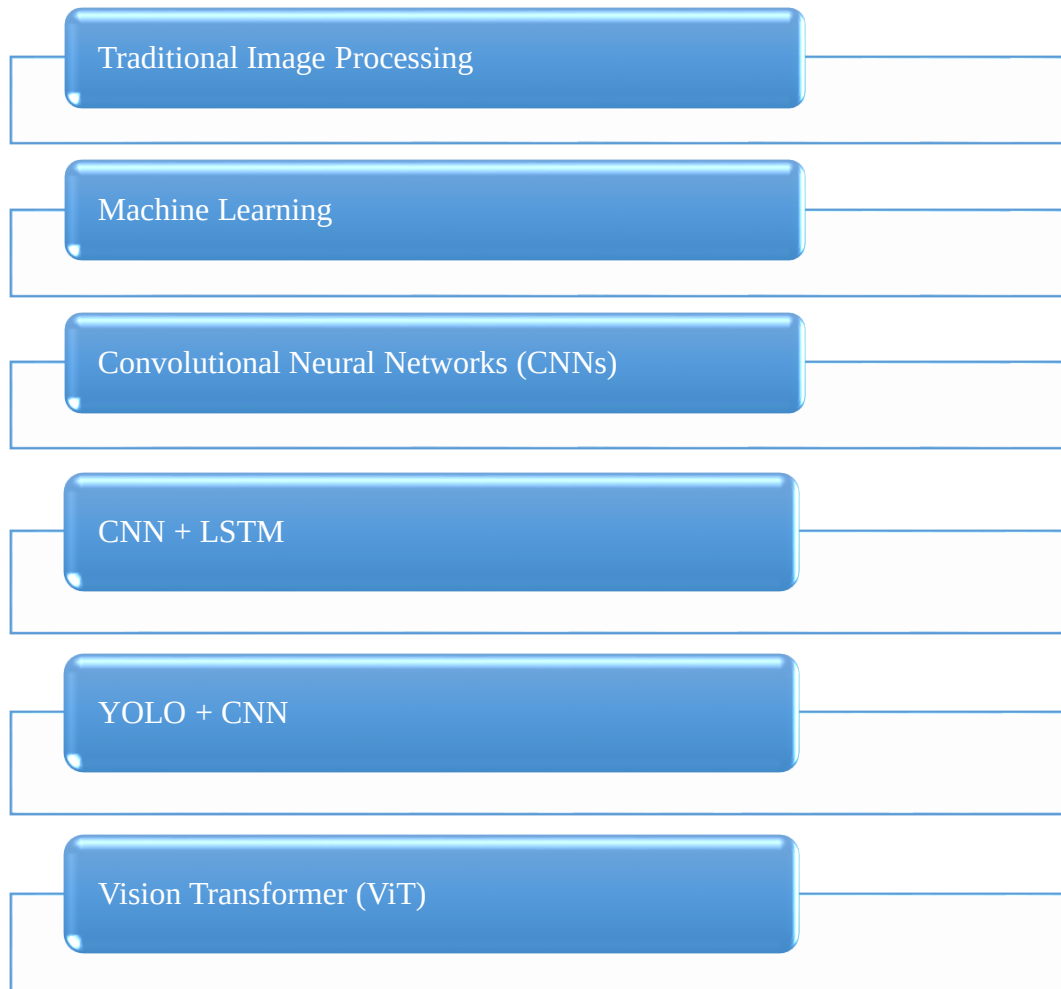
Although existing sign language recognition systems have achieved excellent classification accuracy, many of them still face practical challenges. Some systems require expensive hardware or GPUs, while others struggle under varying environmental conditions or when recognizing dynamic gestures. These limitations reduce their usability in real-world applications.

Table 2.2 Drawbacks of Existing Systems

PROJECT NAME	DRAWBACKS
CNN Based Sign language recognition	Performs well only for static hand gestures and is sensitive to lightning variations.
CNN + LSTM Model	High computational cost and requires a large dataset.
Media Pipe + CNN	Limited vocabulary size and difficulty recognizing complex dynamic gestures.
YOLO + CNN	Performance degrades when multiple hands or people appear simultaneously.
Transfer Learning Approach	Requires GPU acceleration for efficient training and is limited to trained gesture classes.
Vision Transformer (ViT)	Requires very large annotated datasets and high computational power.

Based on the literature survey, it is evident that each technique has its own strengths and weaknesses. CNN-based models are computationally efficient for static sign recognition, whereas Vision Transformers and CNN-LSTM models provide better performance for more complex scenarios but require greater computational resources. These observations guided the selection of a CNN-based approach for the proposed SIGNLEXICON system.

Figure 2.1 Evolution of Sign Language Recognition Techniques



2.4 Research Gap

The literature survey indicates that although many existing systems achieve high recognition accuracy, several limitations remain. Most systems focus either on static alphabet recognition or require powerful hardware for recognizing dynamic gestures. Many applications depend on GPUs for efficient execution and lack simple graphical interfaces that can be easily used by beginners or educational institutions.

Another significant limitation is the dependence on controlled environments. Changes in lighting, camera angle, and background often reduce recognition performance. Furthermore, several existing applications are designed primarily for research purposes rather than practical desktop deployment.

The proposed SIGNLEXICON system addresses these limitations by providing a lightweight desktop application developed using Python, TensorFlow, OpenCV, and Tkinter. The system supports both image-based prediction and real-time webcam recognition using a trained CNN model while presenting prediction confidence through a simple graphical interface.

2.5 Summary

This chapter presented a comprehensive review of existing sign language recognition systems based on various Deep Learning and Computer Vision techniques. Different approaches, including CNN, CNN-LSTM, MediaPipe, YOLO, Transfer Learning, and Vision Transformers, were analyzed along with their datasets, performance, and limitations. The survey revealed that although recent models achieve high recognition accuracy, many still face challenges related to computational complexity, hardware dependency, environmental sensitivity, and usability.

Based on these observations, the proposed SIGNLEXICON system adopts a CNN-based approach integrated with a user-friendly desktop application. The system is designed to provide accurate static sign recognition using both uploaded images and real-time webcam input while maintaining simplicity, accessibility, and practical usability.

CHAPTER III

TRAINING WORK

3.1 Introduction

Industrial training plays a vital role in bridging the gap between theoretical knowledge and practical implementation. It provides students with an opportunity to work on real-world projects while gaining exposure to industrial standards, software development methodologies, and emerging technologies.

The six-week industrial training was carried out at **Solitaire Infosys Pvt. Ltd., Mohali**, where the primary objective was to develop an Artificial Intelligence-based application capable of recognizing American Sign Language (ASL) alphabets. The project, named **SIGNLEXICON**, was developed using Computer Vision and Deep Learning techniques. Throughout the training period, different stages of software development were followed, including requirement analysis, dataset collection, image preprocessing, model development, model training, graphical user interface development, system testing, and result evaluation.

The project was implemented using Python programming language along with TensorFlow, Keras, OpenCV, Tkinter, NumPy, and Pillow libraries. The final application provides two methods of sign recognition: image upload and real-time webcam recognition. The trained Convolutional Neural Network (CNN) model predicts the corresponding alphabet and displays the prediction with its confidence score through a user-friendly desktop interface. The industrial training provided practical exposure to AI application development and enhanced understanding of Deep Learning workflows, Computer Vision techniques, software integration, debugging, and model deployment.

3.2 Company Profile

Solitaire Infosys Pvt. Ltd. is a software development and IT training company located in Mohali, Punjab. The company specializes in software development, web technologies, Artificial Intelligence, Machine Learning, Data Science, Mobile Application Development, and Industrial Training programs for engineering students.

The organization provides practical training through live projects, allowing students to gain industrial experience in software development and modern technologies. During the six-week training period, guidance was provided in project planning, Deep Learning model development, image processing, GUI development, testing, and documentation.

The industrial environment helped in understanding software engineering practices, project management techniques, debugging methods, and collaborative development.

3.3 Training Schedule

Table 3.1 Training Schedule

WEEK	TRAINING ACTIVITIES
Week 1	Company orientation, project selection, Python revision, introduction to TensorFlow and OpenCV.
Week 2	Dataset collection, image preprocessing, dataset organization.
Week 3	CNN architecture design, model development and training.
Week 4	Model evaluation, testing and performance analysis.

Week 5	Development of desktop application using Tkinter.
Week 6	Webcam integration, debugging, GUI improvements, documentation and final testing.

3.4 Project Selection

Several Artificial Intelligence project ideas were discussed during the initial phase of industrial training. Considering the growing importance of accessibility technologies and the practical application of Deep Learning, a Sign Language Recognition System was selected.

The project focuses on recognizing American Sign Language alphabet gestures using Computer Vision and Convolutional Neural Networks. The developed system was named **SIGNLEXICON** and was designed to provide both image-based and real-time webcam recognition through an interactive desktop application.

The selected project provided practical exposure to image classification, neural network training, dataset preprocessing, and software integration while addressing an important social communication challenge.

3.5 Development Methodology

The development of SIGNLEXICON followed a systematic workflow consisting of multiple phases.

Figure 3.1 Development Methodology



3.6 Dataset Collection

The American Sign Language (ASL) Alphabet Dataset was selected for training the CNN model. The dataset contains images representing twenty-six English alphabets along with three additional classes: **space**, **delete**, and **nothing**, resulting in a total of twenty-nine classes.

Each image was carefully organized into separate folders corresponding to its respective class label. The dataset provides sufficient variation in hand orientation and background conditions, enabling the model to learn robust features for accurate classification.

Before training, the dataset was verified to remove corrupted images and ensure proper organization.

Figure 3.2 ASL Alphabet Dataset

Name	Date modified	Type	Size
 A	11-07-2026 23:28	File folder	
 B	11-07-2026 23:32	File folder	
 C	11-07-2026 23:33	File folder	
 D	11-07-2026 23:37	File folder	
 del	12-07-2026 00:24	File folder	
 E	11-07-2026 23:38	File folder	
 F	11-07-2026 23:42	File folder	
 G	11-07-2026 23:43	File folder	
 H	11-07-2026 23:47	File folder	
 I	11-07-2026 23:48	File folder	
 J	11-07-2026 23:52	File folder	
 K	11-07-2026 23:53	File folder	
 L	11-07-2026 23:55	File folder	
 M	11-07-2026 23:58	File folder	
 N	11-07-2026 23:59	File folder	
 nothing	12-07-2026 00:30	File folder	
 O	12-07-2026 00:03	File folder	
 P	12-07-2026 00:04	File folder	
 O	12-07-2026 00:08	File folder	

3.7 Image Preprocessing

Image preprocessing was performed to improve the quality of input images before they were supplied to the CNN model.

The following preprocessing operations were carried out:

- Image resizing to **200 × 200 pixels**
- RGB color conversion
- Brightness enhancement
- Image normalization
- Conversion into NumPy arrays
- Batch preparation for CNN training

These preprocessing steps reduced computational complexity while improving model consistency.

3.8 CNN Model Development

A Convolutional Neural Network (CNN) was selected because of its excellent performance in image classification tasks. CNN automatically extracts important image features without requiring manual feature engineering.

The network consists of multiple convolution layers followed by pooling layers, flattening, dense layers, and a Softmax output layer for multi-class classification.

The model was developed using TensorFlow and Keras libraries.

Figure 3.3 CNN Model Architecture

Layer (type)	Output Shape	Param #
rescaling_1 (Rescaling)	(None, 200, 200, 3)	0
conv2d_3 (Conv2D)	(None, 198, 198, 32)	896
max_pooling2d_3 (MaxPooling2D)	(None, 99, 99, 32)	0
conv2d_4 (Conv2D)	(None, 97, 97, 64)	18,496
max_pooling2d_4 (MaxPooling2D)	(None, 48, 48, 64)	0
conv2d_5 (Conv2D)	(None, 46, 46, 128)	73,856
max_pooling2d_5 (MaxPooling2D)	(None, 23, 23, 128)	0
flatten_1 (Flatten)	(None, 67712)	0
dense_2 (Dense)	(None, 128)	8,667,264
dense_3 (Dense)	(None, 29)	3,741

...

Total params: 8,764,253 (33.43 MB)

...

Trainable params: 8,764,253 (33.43 MB)

...

Non-trainable params: 0 (0.00 B)

3.9 Model Training

The prepared dataset was used to train the CNN model. During training, multiple epochs were executed to allow the model to learn meaningful features from hand gesture images.

The Adam optimizer and categorical cross-entropy loss function were used to optimize model performance.

Table 3.2 Training Parameters

PARAMETER	VALUE
Model	CNN
Image Size	200 X 200
Optimizer	Adam
Loss Function	Categorical Crossentropy
Activation	ReLU, Softmax
Framework	TensorFlow / Keras

3.10 Desktop Application

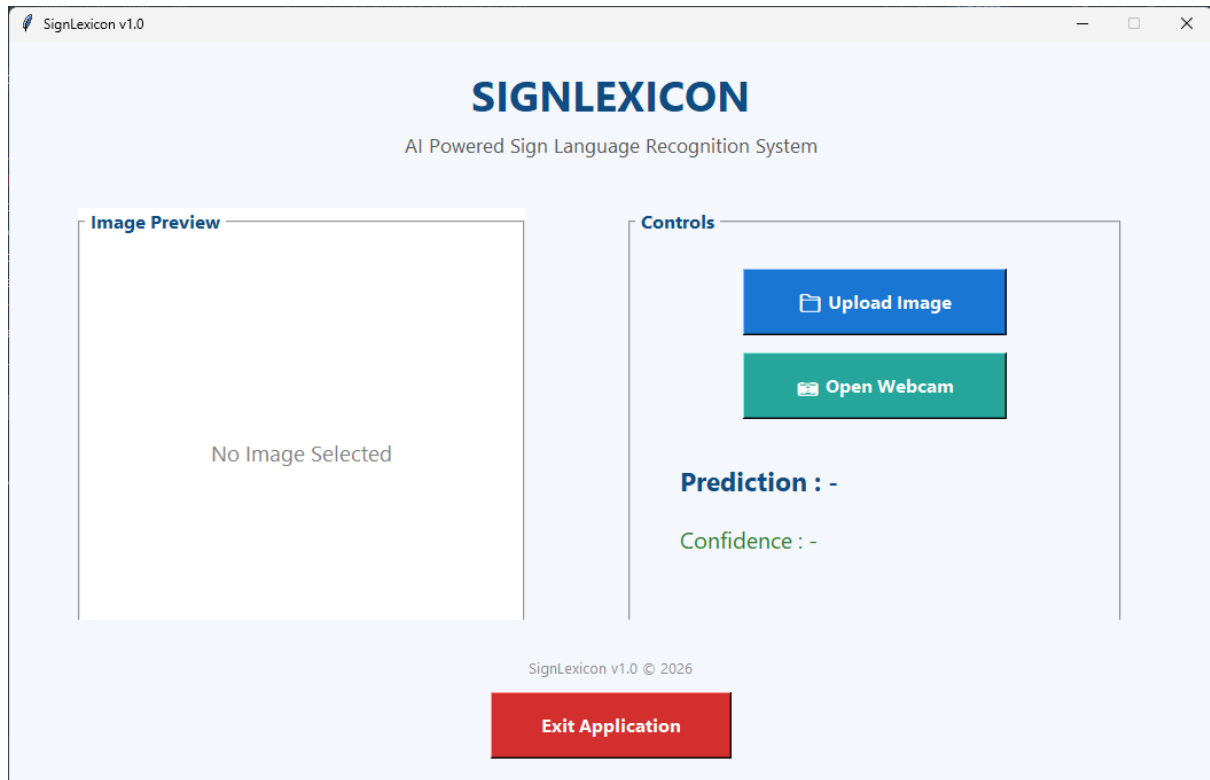
After successful model training, a desktop application named **SIGNLEXICON** was developed using the Tkinter GUI framework.

The application allows users to:

- Upload an image
- Preview the selected image
- Predict the corresponding sign
- Display prediction confidence
- Perform live webcam recognition
- Exit the application safely

The application integrates the trained CNN model with the graphical user interface, making the system easy to operate.

Figure 3.4 SIGNLEXICON Desktop Application



3.11 Webcam Recognition Module

A real-time webcam recognition module was integrated into the application using OpenCV. The webcam continuously captures video frames. A Region of Interest (ROI) is extracted from the center of the frame, preprocessed, and passed to the trained CNN model. The predicted alphabet and confidence score are displayed both on the webcam window and within the desktop application.

3.12 Technologies Used

Table 3.3 Software & Technologies Used

TECHNOLOGY	PURPOSE
Python	Programming Language
TensorFlow	Deep Learning Framework
Keras	CNN Development
OpenCV	Image Preprocessing & Webcam
Tkinter	Desktop GUI
NumPY	Numerical Computation
Pillow	Image Processing
Visual Studio Code	Development Environment

3.13 Training Outcome

The industrial training provided valuable practical experience in Artificial Intelligence, Deep Learning, and Computer Vision. Throughout the six weeks, knowledge gained through classroom learning was applied to the development of a complete software application. Practical experience was obtained in dataset preprocessing, CNN model training, GUI development, debugging, testing, and software documentation.

The successful completion of the SIGNLEXICON project improved programming skills, problem-solving abilities, and understanding of real-world AI application development. The training also enhanced familiarity with industry-standard development tools and software engineering practices.

3.14 Summary

This chapter presented the complete training work carried out during the six-week industrial training at Solitaire Infosys Pvt. Ltd. The chapter discussed the company profile, project selection, development methodology, dataset preparation, CNN model development, model training, GUI development, webcam integration, technologies used, and learning outcomes. The practical implementation of these activities resulted in the successful development of the SIGNLEXICON desktop application for sign language recognition.

CHAPTER IV

EVALUATION OF TRAINING

4.1 Introduction

The evaluation phase is an essential part of any software development project as it verifies whether the developed system satisfies the intended objectives and performs efficiently under different conditions. During the industrial training, the developed application **SIGNLEXICON** was thoroughly tested to evaluate its functionality, prediction accuracy, graphical user interface, and real-time performance. The evaluation was carried out using both uploaded images and live webcam input. Various American Sign Language (ASL) alphabet images were tested to verify the prediction capability of the trained Convolutional Neural Network (CNN). The application was also evaluated for usability, responsiveness, and overall performance under different lighting conditions. The results obtained during testing confirmed that the developed system performs efficiently for static alphabet recognition while providing an easy-to-use desktop interface.

4.2 Testing Methodology

The **SIGNLEXICON** application was tested using multiple approaches to ensure reliable performance.

The following testing methods were adopted:

- Testing using uploaded ASL alphabet images.

- Real-time webcam testing.
- Verification of prediction confidence.
- Graphical User Interface (GUI) testing.
- Functional testing of all buttons.
- Performance testing under different lighting conditions.

The objective of testing was to verify that every module of the application performs correctly and produces accurate predictions.

4.3 Functional Testing

The application consists of several functional modules. Each module was individually tested to verify its operation.

Table 4.1 Functional Testing Results

MODULE	EXPECTED RESULT	OBSERVED RESULT	STATUS
Upload Image	Image opens successfully	Working correctly	Pass
Image Preview	Selected Image Displayed	Working correctly	Pass
CNN Prediction	Predict correct alphabet	Working correctly	Pass
Confidence Score	Display prediction confidence	Working correctly	Pass
Webcam Recognition	Real – time prediction.	Working correctly	Pass
Exit Button	Close application.	Working correctly	Pass

4.4 Experimental Results

The developed SIGNLEXICON application was tested using different ASL alphabet images from the dataset. The trained CNN model successfully recognized the uploaded images and displayed the corresponding prediction along with the confidence score. Similarly, the webcam module was tested by performing live hand gestures inside the Region of Interest (ROI). The application successfully recognized the gestures in real time and updated the prediction continuously.

The following screenshots demonstrate the successful execution of different modules of the application.

Figure 4.1 Home Screen of SIGNLEXICON

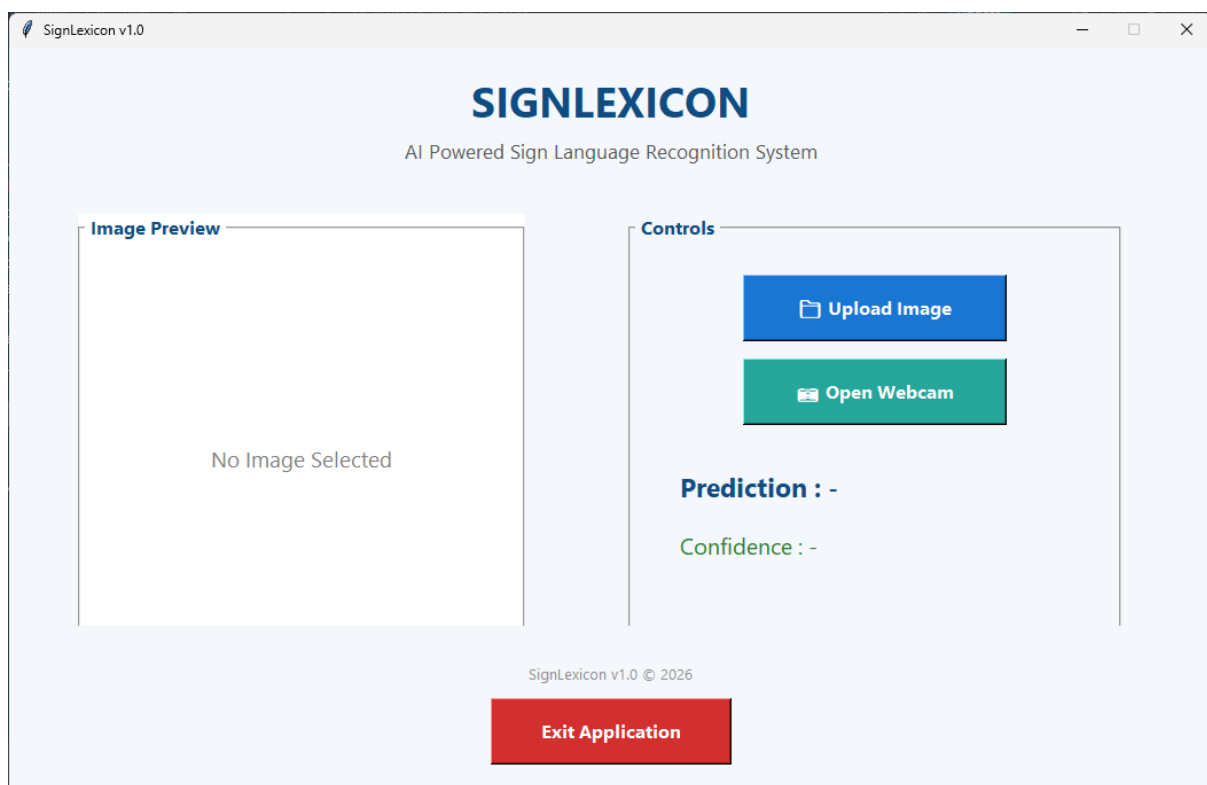


Figure 4.2 Image Upload Prediction

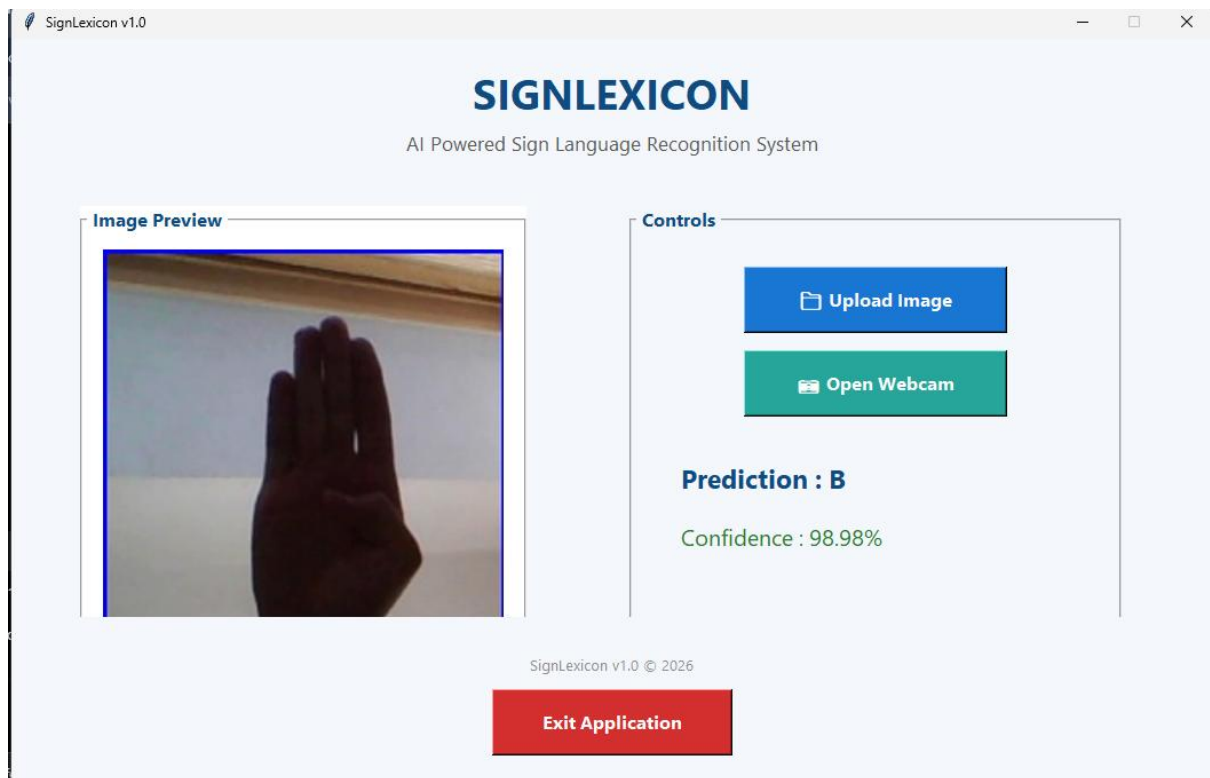


Figure 4.3 Webcam Recognition

```

cv2.imshow("SignLearn Webcam", frame)

key = cv2.waitKey(1) & 0xFF

if key == ord('s'):
    cv2.imwrite("webcam_test.jpg", roi)
    print("Image saved!")

elif key == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

```

Python

```

...

Top Predictions:
J : 0.3240
R : 0.1937
space : 0.0639
K : 0.0615
Y : 0.0492

Top Predictions:
R : 0.2999
space : 0.1744
J : 0.1336
K : 0.1005
U : 0.0567

```

4.5 Performance Evaluation

The overall performance of the developed application was evaluated on the basis of recognition capability, usability, responsiveness, and prediction accuracy.

Table 4.2 Performance Evaluation

PARAMETER	RESULT
Recognition Technique	CNN
Supported Classes	29
Input Method	Image Upload & Webcam
Prediction Output	Alphabet + Confidence
GUI Performance	Smooth
Real – Time Recognition	Successful
User Interface	User Friendly

4.6 Sample Prediction Results

The following table presents some sample predictions obtained during testing.

Table 4.3 Sample Prediction Results

INPUT IMAGE	EXPECTED OUTPUT	PREDICTED OUTPUT	RESULT
A	A	A	Correct
B	B	B	Correct

M	M	M	Correct
Y	Y	Y	Correct
Space	Space	Space	Correct

Figure 4.4 Sample Prediction Results

```

Generate + Code + Markdown

img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
prediction = model.predict(img_array)

[11] Python
... 1/1 ————— 0s 70ms/step

predicted_index = np.argmax(prediction)
predicted_class = class_names[predicted_index]
print("Predicted Sign:", predicted_class)

[12] Python
... Predicted Sign: B

confidence = np.max(prediction)
print(f"Confidence: {confidence*100:.2f}%")

[13] Python
... Confidence: 98.98%

```

4.7 Discussion

The experimental results indicate that the CNN model performs effectively for static American Sign Language alphabet recognition. The application successfully classified uploaded images and real-time webcam input while displaying the corresponding confidence score. The desktop interface developed using Tkinter proved to be simple and easy to operate. The image upload

functionality and webcam module worked as expected without major issues. During testing, the model produced higher confidence values for clear hand gestures captured under adequate lighting conditions.

However, it was observed that prediction confidence decreased slightly when the hand was partially outside the Region of Interest or when illumination was poor. Despite these minor challenges, the overall performance of the developed system was satisfactory and met the objectives defined at the beginning of the project.

4.8 Challenges Faced During Training

During the development of SIGNLEXICON, several practical challenges were encountered.

Some of the major challenges included:

- Understanding Deep Learning concepts and CNN architecture.
- Preparing and organizing the dataset.
- Integrating the trained CNN model with the desktop application.
- Displaying uploaded images correctly inside the GUI.
- Integrating OpenCV webcam with the Tkinter application.
- Managing Python library dependencies.
- Improving the alignment and appearance of the graphical interface.
- Debugging prediction and confidence display issues.

These challenges were successfully addressed through continuous testing, debugging, and guidance received during the industrial training.

4.9 Skills Acquired During Training

The industrial training contributed significantly to technical and professional development.

The following skills were acquired:

- Python Programming
- TensorFlow and Keras
- Computer Vision using OpenCV
- CNN Model Development
- Image Processing
- GUI Development using Tkinter
- Model Testing and Evaluation
- Software Debugging
- Technical Documentation
- Problem-Solving Skills

4.10 Summary

This chapter presented the evaluation of the SIGNLEXICON application developed during the industrial training. The developed system was tested using uploaded images as well as real-time webcam input. Functional testing confirmed that all modules operated successfully, while experimental results demonstrated satisfactory recognition performance. The evaluation also highlighted the practical skills acquired, challenges encountered, and the successful achievement of the project objectives.

CHAPTER V

CONCLUSION & FUTURE SCOPE OF TRAINING

5.1 Introduction

The industrial training provided an excellent opportunity to apply theoretical knowledge in a practical environment. During the six-week training at **Solitaire Infosys Pvt. Ltd., Mohali**, a complete Artificial Intelligence-based Sign Language Recognition System named **SIGNLEXICON** was successfully designed and developed.

The project combined Computer Vision and Deep Learning techniques to recognize American Sign Language (ASL) alphabets through both image upload and real-time webcam input. The successful completion of this project strengthened practical knowledge in software development, image processing, neural networks, and graphical user interface design. This chapter presents the conclusion of the work carried out during the training, the knowledge gained, limitations of the developed system, and possible future enhancements.

5.2 Conclusion

The primary objective of this industrial training was to develop a desktop-based Sign Language Recognition System capable of recognizing American Sign Language (ASL) alphabets using Computer Vision and Deep Learning techniques. This objective was successfully achieved through the development of **SIGNLEXICON**.

The project was implemented using Python and various open-source libraries such as TensorFlow, Keras, OpenCV, NumPy, Pillow and Tkinter. A Convolutional Neural Network (CNN) was trained to classify ASL alphabet images, and the trained model was integrated into a desktop application that supports both image upload and real-time webcam recognition. The developed application provides accurate alphabet prediction along with confidence scores, making it easy for users to interpret the model's output. The graphical user interface was designed to be simple, interactive, and user-friendly. The system demonstrated satisfactory performance during testing and successfully fulfilled the objectives established at the beginning of the project.

Overall, the industrial training enhanced practical knowledge of Artificial Intelligence, Deep Learning, Computer Vision, software development, debugging, and technical documentation. The experience gained during the training will be valuable for future academic projects as well as professional work in the field of Artificial Intelligence and Robotics.

5.3 Learning Outcomes

The industrial training significantly contributed to both technical and professional development. The following learning outcomes were achieved:

- Practical implementation of Artificial Intelligence concepts.
- Understanding of Computer Vision techniques.
- Development of image classification models using Convolutional Neural Networks.
- Image preprocessing using OpenCV.
- Deep Learning model development using TensorFlow and Keras.
- Development of desktop applications using Python Tkinter.

- Integration of AI models with graphical user interfaces.
- Real-time webcam processing and prediction.
- Software testing and debugging.
- Technical documentation and report preparation.
- Improved analytical thinking and problem-solving skills.
- Better understanding of software development life cycle in an industrial environment.

5.4 Limitations of the Proposed System

Although the developed SIGNLEXICON application performs efficiently for static sign recognition, certain limitations still exist.

The major limitations are:

- The application currently recognizes only American Sign Language (ASL) alphabets.
- Dynamic gestures and continuous sentence recognition are not supported.
- Performance may decrease under poor lighting conditions.
- Background clutter may affect prediction accuracy.
- The system recognizes only one hand at a time.
- Accurate prediction requires the hand to remain within the Region of Interest (ROI).
- The application is currently available only as a desktop application.
- The vocabulary is limited to the trained dataset.

5.5 Future Scope

The proposed SIGNLEXICON system provides a strong foundation for future research and development. Several improvements can further enhance the performance and usability of the application.

Future enhancements may include:

- Recognition of complete words and sentences instead of individual alphabets.
- Support for dynamic sign language recognition using video sequences.
- Integration of speech synthesis to convert recognized signs into spoken language.
- Development of Android and iOS mobile applications.
- Support for Indian Sign Language (ISL), British Sign Language (BSL), and other regional sign languages.
- Cloud-based deployment for remote accessibility.
- Improved recognition under different lighting and background conditions.
- Integration of Natural Language Processing (NLP) for sentence formation.
- Multi-user recognition capability.
- Optimization of the AI model for faster real-time performance on low-end devices.

5.6 Industrial Training Experience

The six-week industrial training at **Solitaire Infosys Pvt. Ltd.** was a highly valuable learning experience. It provided practical exposure to real-world software development and Artificial Intelligence applications. Working on a live project helped bridge the gap between classroom learning and industry practices.

Throughout the training, guidance from mentors and continuous practice improved programming skills, debugging techniques, project management abilities, and confidence in solving technical challenges. The experience also emphasized the importance of teamwork, documentation, testing, and maintaining software quality throughout the development process.

The successful completion of the SIGNLEXICON project marks an important milestone in the practical understanding of Artificial Intelligence, Deep Learning, and Computer Vision technologies.

5.7 Final Conclusion

The SIGNLEXICON project demonstrates the successful application of Artificial Intelligence and Deep Learning for sign language recognition. By combining Computer Vision techniques with a Convolutional Neural Network, the developed system provides an efficient and user-friendly solution for recognizing American Sign Language alphabets.

The project successfully integrates image processing, real-time webcam recognition, and an interactive desktop application into a single platform. It contributes toward improving communication accessibility and demonstrates how Artificial Intelligence can be used to address real-world challenges.

The industrial training not only enhanced technical competence but also developed professional skills such as analytical thinking, problem-solving, software development, and technical documentation. The knowledge and experience gained during this training will serve as a strong foundation for future research and career opportunities in Artificial Intelligence, Robotics, and Machine Learning.

5.8 Summary

This chapter presented the conclusion of the industrial training, highlighted the knowledge and skills acquired during the project, discussed the limitations of the developed system, and identified potential future enhancements. The successful completion of SIGNLEXICON demonstrates the practical application of Computer Vision and Deep Learning in developing an effective sign language recognition system and reflects the valuable learning achieved during the industrial training.

APPENDICES

Appendix A

Hardware & Software Requirements

The SIGNLEXICON application was developed and tested using the following hardware and software configuration.

Table A.1 Hardware Requirements

COMPONENT	SPECIFICATION
Processor	Intel Core i3 or higher
RAM	8 GB Minimum
Storage	10 GB Free Disk Space
Webcam	Integrated/External USB Webcam
Display	1366 × 768 Resolution or Higher

Table A.2 Software Requirements

SOFTWARE	VERSION
Windows Operating System	Windows 10/11
Python	3.x
TensorFlow	Latest Compatible Version

OpenCV	Latest Compatible Version
Keras	Latest Compatible Version
NumPY	Latest Compatible Version
Pillow	Latest Compatible Version
Visual Studio Code	IDE Used

APPENDIX B

USER MANUAL

This appendix provides the basic operating procedure of the SIGNLEXICON application.

Step 1

Launch the SIGNLEXICON desktop application.

Step 2

Choose one of the following options:

- Upload Image
- Open Webcam

Step 3

For image recognition:

- Click **Upload Image**.
- Select an ASL image.
- Wait for prediction.
- View the predicted alphabet and confidence score.

Step 4

For webcam recognition:

- Click **Open Webcam**.
- Place your hand inside the blue Region of Interest (ROI).
- Wait for prediction.
- Press **Q** to close the webcam.

Step 5

Click the **Exit** button to close the application.

Notes

- Ensure sufficient lighting.
- Keep the hand inside the ROI.
- Avoid background clutter.
- Use clear hand gestures for better accuracy.

Appendix C

Project Achievements

The major achievements of the SIGNLEXICON project are summarized below:

- Successfully developed a desktop-based Sign Language Recognition System.
- Implemented a Convolutional Neural Network for image classification.
- Integrated Deep Learning with a graphical user interface.
- Developed both image upload and real-time webcam recognition modules.
- Displayed prediction confidence for every recognized sign.
- Successfully tested the application using multiple ASL alphabet images.
- Gained practical experience in Computer Vision and Deep Learning.
- Improved software development, debugging, and documentation skills during industrial training.

REFERENCES

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [2] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [3] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson Education, 2018.
- [4] A. Kaehler and G. Bradski, *Learning OpenCV 4: Computer Vision with Python*. Sebastopol, CA, USA: O'Reilly Media, 2019.
- [5] F. Chollet, *Deep Learning with Python*, 2nd ed. Manning Publications, 2021.
- [6] TensorFlow Developers, **TensorFlow Documentation**. Available: <https://www.tensorflow.org/>
- [7] OpenCV Team, **OpenCV Documentation**. Available: <https://opencv.org/>
- [8] Python Software Foundation, **Python Documentation**. Available: <https://www.python.org/>
- [9] Pillow Developers, **Pillow Documentation**. Available: <https://python-pillow.org/>
- [10] NumPy Developers, **NumPy Documentation**. Available: <https://numpy.org/>
- [11] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv preprint arXiv:1804.02767*, 2018.
- [12] Kaggle, **ASL Alphabet Dataset**. Available: <https://www.kaggle.com/>

